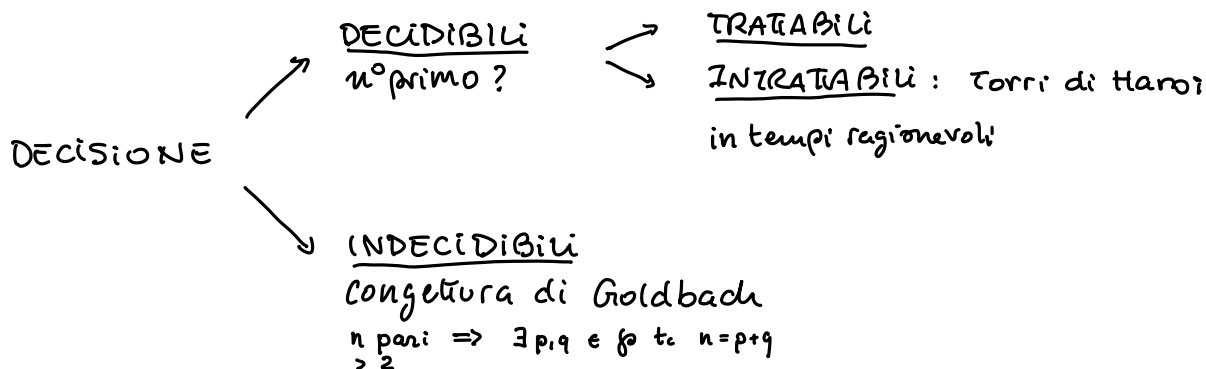


A TIPOLOGIE di PROBLEMI D R i V e O n

- 1) decisione $\Rightarrow$ sì/no : n^o è primo?
- 2) ricerca $\Rightarrow$ se $\exists$ e qual è: ciclo hamiltoniano?
ordinamento
- 3) verifica $\Rightarrow$ sol valida: 8 regine ...?
Sudoko ...?
- 4) ottimizzazione $\Rightarrow$ se $\exists$ e qual è: cammini min/max
migliore

A1 PROBLEMI di DECISIONE



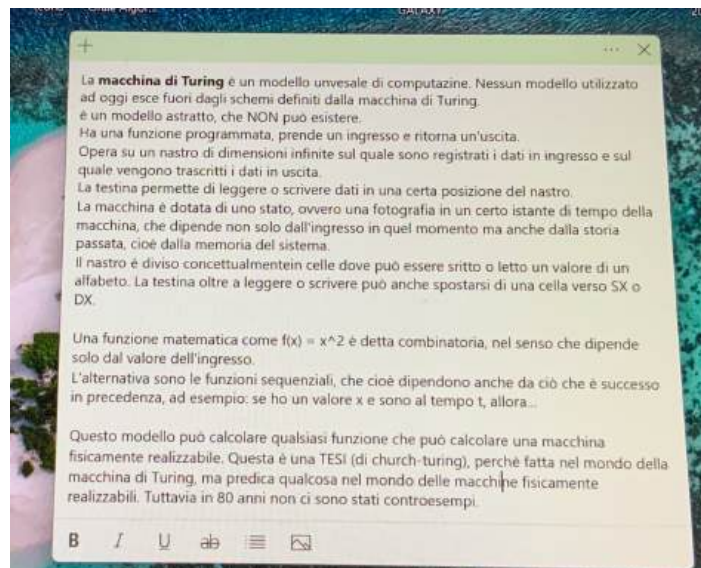
DECISIONE: decidibile trattabile

```

int ISprime (int n) {
    if (n == 1) return 1;

    int fact = 2; 2  $\rightarrow \sqrt{n}$ 
    while (fact <= sqrt(n)) {
        if (n % fact == 0)
            return 1;
        fact++;
    }
    return 0;
}

```



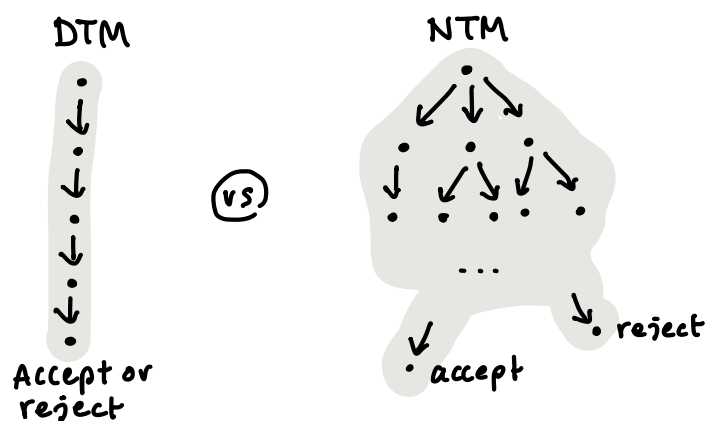
① MACCHINA di TURING : modello matematico di macchina computazionale generica

MACCHINA di TURING MdT

- NASTRO ∞ : legge e scrive simboli
- simboli e ALFABETO finito
- n° di stati finiti $\{ \text{START}, \dots, \text{END} \}$

MACCHINA di TURING NON DETERMINISTICA

- Prova più strade contemporaneamente per risolvere un problema



② CLASSI di COMPLESSITA'

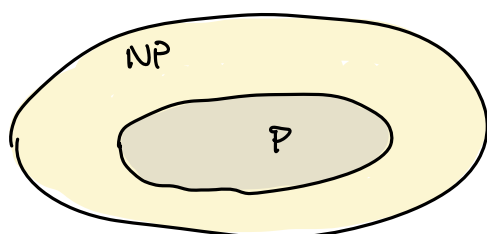
1) **P** = problemi risolvibili DTM con tempo polinomiale

TIME: $\mathcal{O}(n^2)$ max

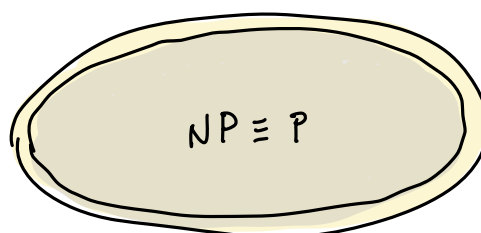
2) **NP** = _____ NTM con tempo polinomiale
or
_____ DTM con tempo esponenziale

TIME: $\mathcal{O}(n^2)$ min

③ verifica di una sol e' polinomiale



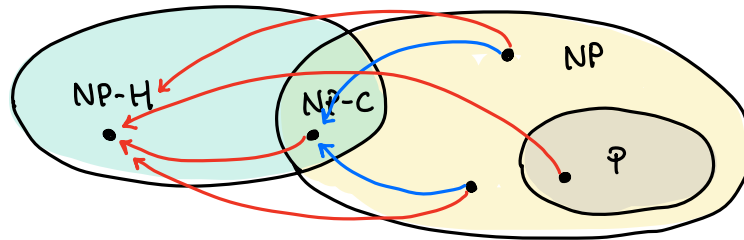
PROBABILE



IMPROBABILE

3) NP-Complete $\left\{ \begin{array}{l} 1. \in NP \\ 2. \text{ogni altro problema NP} \end{array} \right. \xrightarrow[\text{POLINOMIALE}]{\text{TRASFORM.}} \text{eno}$

4) NP-Hard : ogni problema in NP, NP-C, P $\xrightarrow[\text{POLINOMIALE}]{\text{TRASFORM.}} \text{eno}$



P: PRIMALITÀ
CONNETTIVITÀ GRAFI

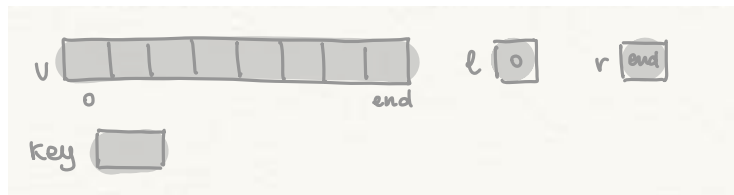
NP: FATTORIZZAZIONE

NP-C: CIRCUITO
CICLO di HAMILTON

NP-H: PERMANENZE di
una MATRICE

③ ALGORITMI di RICERCA su VETTORI

1) int Linear SEARCH (int *v, int l, int r, int key); $\mathcal{O}(n)$



2) BINARIA o DICOTOMICA

int Bin SEARCH (int *v, int l, int r, int key); $\mathcal{O}(\log_2 n)$



TERMINAZIONE
 $n, \frac{n}{2}, \frac{n}{4} \Rightarrow \frac{n}{2^i} = 1$

```
int Bin SEARCH (int *v, int l, int r, int key) {
```

```
    int m;
    while (l <= r) {
        m = (l+r)/2;
```

```
int Bin SEARCH-R (int *v, int l, int r, int key) {
```

```
    int m = (l+r)/2;
```

```
    // COND di TERM
```

```
    if (l > r)
```

```

if (v[m] == key)
    return m;

if (v[m] < key)
    l = m + 1;

else (v[m] > key)
    r = m - 1;
}
return -1;

```

```

return (m = -1);

if (v[m] == key)
    return m;

// PASSO RICORSIONE
if (v[m] > key)
    BinSEARCH-R(v, l, m-1, key);
else
    BinSEARCH-R(v, m+1, r, key);

return m;

```

④ NOTAZIONE ASINTOTICA

• $\mathcal{O}$ "o-grande"

$$T(n) = \mathcal{O}(g(n)) \iff \exists c > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0 :$$

$$0 \leq T(n) \leq c g(n)$$

$g(n)$ = limite superiore lasco

• Ω "omega-grande"

$$T(n) = \Omega(g(n)) \iff \exists c > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0 :$$

$$0 \leq c g(n) \leq T(n)$$

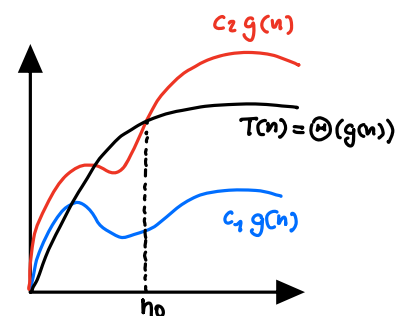
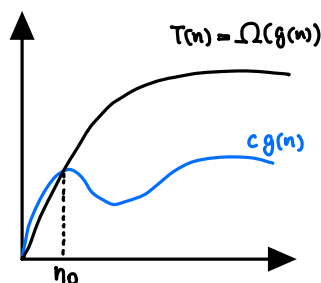
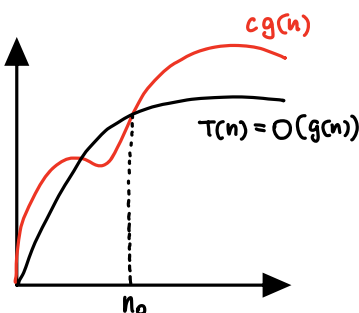
$g(n)$ = limite inferiore lasco

• Θ "theta-grande"

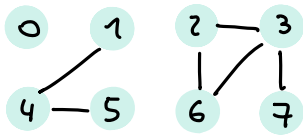
$$T(n) = \Theta(g(n)) \iff \exists c_1, c_2 > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0 :$$

$$0 \leq c_1 g(n) \leq T(n) \leq c_2 g(n)$$

$g(n)$ = limite asintotico stretto



Componenti connesse: nodi mutualmente raggiungibili

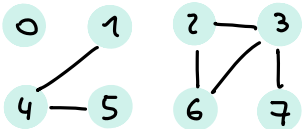


SCC: $\{0\}$ $\{1, 4, 5\}$ $\{2, 3, 6, 7\}$

2 poteri: $\left\{ \begin{array}{l} 1. \text{ Non abbiamo a disposizione il grafo} \\ 2. \text{ Si lavora 1 coppia alla volta (on-line)} \\ 3. \text{ Inizialmente } V \text{ insiemi unitari} \end{array} \right.$

Operazioni: $\left\{ \begin{array}{l} 1. \text{ FIND: trova l'insieme a cui } \epsilon \\ 2. \text{ UNION: unisce 2 insiemi} \end{array} \right.$

(A) Quick FIND

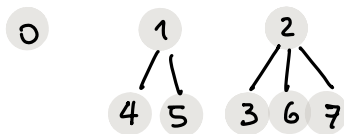


id

0	1	2	2	1	1	2	2
---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7

$\left\{ \begin{array}{l} \bullet \text{ CONNESSIONE: } id[i] = id[j] \Leftrightarrow i \text{ conn } j \\ \bullet \text{ FIND: } id[i] \quad O(1) \\ \bullet \text{ UNION: scansione di id } \quad O(n) \end{array} \right.$



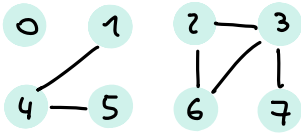
`void Quickfind_CONNECT (int *id, int v, int w, int N) {` $O(n)$

```
    if (id[v] == id[w]) {
        printf("Already connected\n");
        return;
    }
```

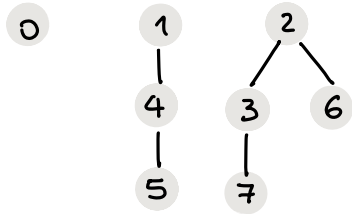
```
    for (int i=0; i<N; i++)
        if (id[i] == id[v])
            id[i] = id[w];
```

```
}
```

(B) QUICK UNION



id	0	1	2	2	1	4	2	3
	0	1	2	3	4	5	6	7



- CONNESSIONE: $id[...id[i]] = id[...id[j]]$
 - FIND: percorre una catena $O(n)$
 - UNION: attribuzione $O(1)$
- *significativa la lunghezza!!!*

```
void Quickunion-CONNECT (int *id, int v, int w, int N) {
```

```
    int i=v;
    while (id[i] != i)
        i = id[i];
```

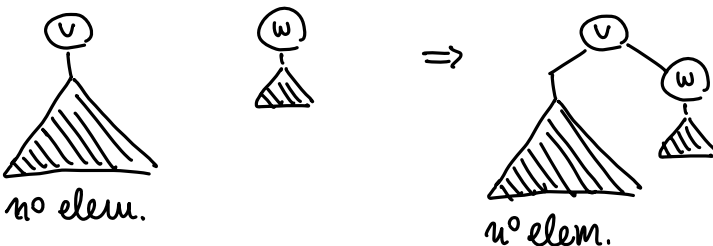
```
    int j=w;
    while (id[j] != j)
        j = id[j];
```

```
    if (id[i] == id[j]) {
        printf("Already connected\n");
        return;
    }
```

```
    id[i] = id[j];
```

(C) WEIGHTED QUICK UNION

Ottimizzazione: OB "Ridurre lunghezza catena"



```
void WeightedQuickunion-CONNECT (int *id, int *sz, int v, int w, int N) {
```

```
    int i=v;
    while (i != id[i]) i = id[i];
```

```
    int j=w;
    while (j != id[j]) j = id[j];
```

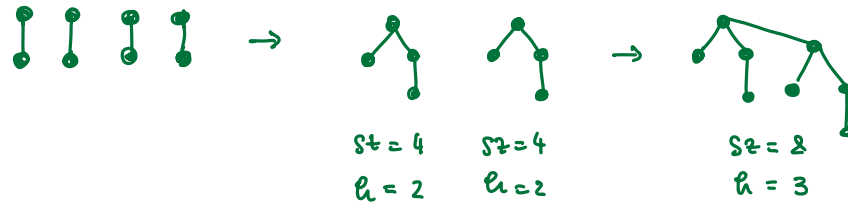
```
    if (id[i] == id[j]) {
        printf("Already connected\n");
        return;
    }
```

```

3
if (sz[i] <= sz[j])
  ↓
  id[i] = id[j];
  sz[j] += sz[i];
}
else if
  ↓
  id[j] = id[i];
  sz[i] += sz[j];
}
3

```

CASO PEGGIORE : unisco sempre alberi $\ominus$ in sz.



$$\Rightarrow \begin{cases} sz = 2^h \\ h = \log_2 sz \end{cases} \quad \text{dove } sz \text{ finale} = n$$

$$\Rightarrow O(\log_2 n)$$

CAPITOLO 03

ALGORITMI di ORDINAMENTO

Insertion SORT
 Bubble SORT
 Selection SORT

} semplici, iterativi, confronto

Shell SORT

} sequenza

Merge SORT
 Quick SORT
 Heap SORT

} complessi, ricorsivi, confronto

Counting SORT

} ipotesi restrittive sui dati, calcolo

$O(n^2)$

$O(n^{3/2})$

$O(n \log n)$

$O(n)$

void Insertion SORT (Item *v, int N) {

$O(n^2)$

Item tmp; int j;

for (int i=1; i < N; i++) {

tmp = v[i];

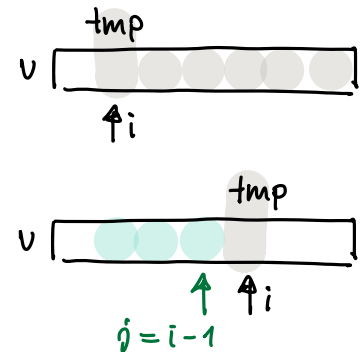
j = i-1;

while (j >= 0 && ITEMgt(tmp, v[j])) {

v[j+1] = v[j];

j--;

v[j+1] = tmp;



• In WCC : bastano v[] + variabili ausiliarie

• STABILE : $2_1 \ 2_2 \ 1 \rightarrow 2_1 \ 2_2 \ 1 \rightarrow 1 \ 2_1 \ 2_2$

void Bubble SORT (Item *v, int N) {

$O(n^2)$

Item tmp;

for (int i=0; i < N-1; i++) {

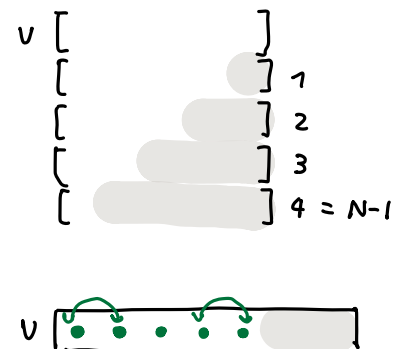
for (int j=0; j < N-1-i; j++) {

if (ITEMgt (v[j], v[j+1])) {

tmp = v[j];

v[j] = v[j+1];

v[j+1] = tmp;



• In LOCO

• STABILE : $2_1 \ 2_2 \ 1 \rightarrow 2_1 \ 1 \ 2_2 \rightarrow 1 \ 2_1 \ 2_2$

void Selection SORT (Item * v, int N) { $O(n^2)$

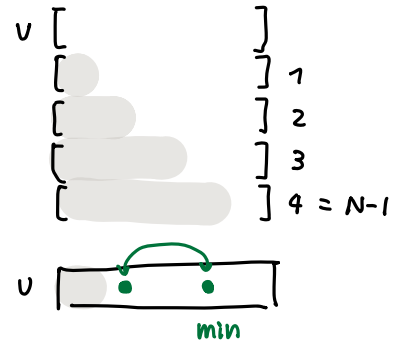
```

int min;
Item tmp;

for (int i=0; i<N-1; i++) {
    //TROVO MIN
    min=i;
    for (j=i+1; j<N-1; j++)
        if (ITEMlt(v[j], v[min]))
            min=j;

    //SWAP
    if (min!=i) {
        tmp = v[i];
        v[i] = v[min];
        v[min] = tmp;
    }
}

```



• In LOCO

• NON STABILE : $2_1 \ 2_2 \ 1 \rightarrow 1 \ 2_2 \ 2_1$

void Shell SORT (Item * v, int N) { $O(n^{3/2})$

```

Item tmp;

//IMPOSTO h: seq di KNUTh 1, 4, 13, ...
int h=1;
while (h<N)
    h=3*h+1;

//INSERTION SORT i-esimo
while (h>=1) {
    for (i=h; i<N; i++) {
        tmp = v[i];
        j=i;
        while (j>=h && ITEMlt(tmp, v[j-h])) {
            v[j] = v[j-h];
            j-=h;
        }
        v[j] = tmp;
    }
    h=h/3;
}

```

Insertion SORT di v già ordinato $\Rightarrow O(n) !!!$

• In LOCO

$$z_1 \quad 1 \quad z_3 \quad z_4 \quad z_5 \quad z_2$$

A diagram illustrating a binary tree structure. On the left, a vertical curly brace is labeled $\log_2 n$. To the right of the brace, there are three levels of light blue rounded rectangular nodes. The top level has one node. The middle level has two nodes, each connected to the top node by a double-headed arrow. The bottom level has four nodes, each connected to one of the middle nodes by a double-headed arrow.

// RICOMBINA

```
void Merge (Item *v, Item *w, int l, int m, int r) {
```

```
    int i = l;      // ind sottov sx
    int j = m+1;    // ind sottov dx
```

```
    for (int k = l; k <= r; k++) {
```

```
        if (i > m) // esaurito sottov sx
```

```
            w[k] = v[j++];
```

```
        else if (j > r) // esaurito sottov dx
```

```
            w[k] = v[i++];
```

```
        else if (ITEMlt(v[i], v[j]) || ITEMeq(v[i], v[j]))
```

```
            w[k] = v[i++];
```

```
        else
```

```
            w[k] = v[j++];
```

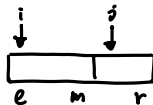
```
    }
```

```
    // ricopia w in v
```

```
    for (int k = l; k <= r; k++)
```

```
        v[k] = w[k];
```

```
}
```



• NON in LOCO : usa w ausiliario

• STABILE : se eq $\Rightarrow$ pesca da sottov sx

// WRAPPER

```
void QuickSORT (Item *v, int N) {
```

```
    int l = 0, r = N-1;
```

```
    QuickSORT-R (v, l, r);
```

```
}
```

// RICORSIVA

```
void QuickSORT-R (Item *v, int l, int r) {
```

```
    // COND. di TERM.
```

```
    if (r <= l) return;
```



```
    // PARTIZIONAMENTO
```

```
    int m = partition(v, l, r);
```

```
    QuickSORT-R (v, l, m-1);
```

```
    QuickSORT-R (v, m+1, r);
```

```
    return;
```

```
}
```

// PARTIZIONE

```
int partition (Item *v, int l, int r) {
```

```
    Item pivot = v[r];
```

```
    int i = l, j = r-1;
```

```
    while (i < j) {
```

```
        while (ITEMlt(v[i], pivot))
```

```
            i++;
```

```
        while (ITEMgt(v[j], pivot))
```

```
            j--;
```

```
        if (i < j)
```

```
            Swap (v, i, j);
```

```
    }
    Swap (v, i, r);
```

```
    return i;
```



operazioni per livello: n

CASO PEGGIORE $\overbrace{1 \dots 1}^{n-1}$

$$T(n) = T(n-1) + n$$

$$T(1) = 1$$

$$\hookrightarrow T(n) = n + n-1 + n-2 + \dots + 3 + 2 + 1$$

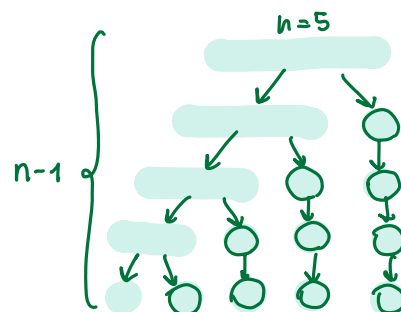
$$= \sum_{i=0}^{n-1} (n-i) = \frac{n}{2} * (n-1) =$$

$$= O(n^2).$$

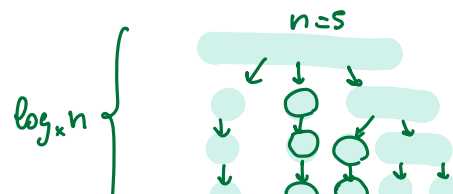
Perché non si abbia un vett
ordinato in senso inverso

$$\rightarrow O(n \log n)$$

= caso migliore $\overbrace{1 \dots 1}^{n/2} \quad \overbrace{1 \dots 1}^{n/2}$



operazioni per lv: n



3



• 2N LOG

• NON STABILE

operations per $lv : n$

- OB
- Risoluzione sotto-problemi $\ominus$ ma più piccoli
 - Espressione del paradigma Divide Et IMPERA
 - TERMINAZIONE $\rightarrow$ banale $\{es: \text{insiemi di 1 el}\}$
 $\rightarrow$ esaurimento scelte lecite $\{es: \text{n° vert grafo}\}$

DECREASE $a=1$

DIVIDE: $a>1$ problema $T(n) \rightarrow T(n')$ dove $n' < n$

IMPERA: risol problema elementare //COND di TERM

COMBINA: ricostr sol complessiva con sol parziali

EQUAZIONI ALLE RICORRENZE

Problema $T(n)$ espresso come:

- $D(n)$: costo della divisione
- $C(n)$: costo della ricombinazione
- a : n° di sottoproblemi a ogni passo
 $a=1$, ricorsione lineare
 $a>1$, ricorsione multi-via
- b : fattore di riduzione $\rightarrow \frac{n}{b}$ dim di sottoprobl.

$$\Rightarrow T(n) = D(n) + a T\left(\frac{n}{b}\right) + C(n)$$

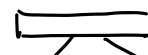
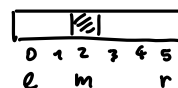
$$T(1) = 1$$

FUNZIONI RICORSIVE IMMEDIATE

MAX-R

int Max(int *v, int N) {

int l=0, r=N-1;
 return Max-R(v, l, r);



3

```
int max(int *v, int l, int r) {
```

// DIVIDE

```
int m = (l+r)/2;
```

// IMPERA

```
if (l == r)
```

```
return v[l];
```

// RICORSIONE

```
int max_sx = max(v, l, m);
```

```
int max_dx = max(v, m+1, r);
```

// COMBINE

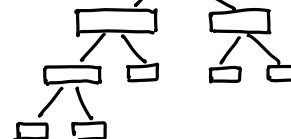
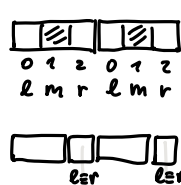
```
if (max_sx > max_dx)
```

```
return max_sx;
```

```
else
```

```
return max_dx;
```

3



EQ. RICORRENTE

$$\left. \begin{array}{l} D(n) = O(n) \\ C(n) = O(1) \\ a = 2 \quad b = 2 \end{array} \right\}$$

$$\boxed{\begin{array}{l} T(n) = 2T(\frac{n}{2}) + 1 \\ T(1) = 1 \end{array}}$$

$$\rightarrow T(n) = 2T(\frac{n}{2}) + 4 + 2 + 1 \approx$$

$$\sum_{i=0}^{\log_2 n} 2^i = \frac{2^{\log_2 n + 1} - 1}{2 - 1} = O(n).$$

FATTORIALE n!

$$\left\{ \begin{array}{l} n! = n(n-1)! \\ 0! = 1 \end{array} \right.$$

```
int fact(int n) {
```

```
if (n == 0) return 1;
```

```
return n * fact(n-1);
```

3

$$T(n) = T(n-1) + 1$$

$$n-1 = 1$$

$$T(1) = 1$$

$$\rightarrow T(n) = T(n-3) + 1 + 1 + 1 = \sum_{i=0}^{n-1} 1 = O(n).$$

FIBONACCI

0 1 1 2 3 5 8

$$Fib(n) = Fib(n-1) + Fib(n-2)$$

$$Fib(1) = 1$$

$$Fib(0) = 0$$

```
int Fib(int n) {
```

// COND di TERM

```
if (n == 1) return 1;
```

```
if (n == 0) return 0;
```

// DECREASE + RICORSIONE

```
int fib_n2 = Fib(n-2);
```

```
int fib_n1 = Fib(n-1);
```

// COMBINA

```
return fib_n2 + fib_n1;
```

}

$$\rightarrow n-i=1 \rightarrow i=n-1$$

$$D(n) = \Theta(1)$$

$$C(n) = \Theta(1)$$

$$a = 2 \quad k = 1 \quad k' = 2$$

$$\boxed{\begin{array}{l} T(n) = T(n-1) + T(n-2) + 1 \\ T(0) = 1 \\ T(1) = 1 \end{array}}$$

$$\rightarrow T(n) \approx 2T(n-1) + 1 = 2[2[2T(n-3) + 1] + 1] + 1 \approx$$

$$4 + 2 + 1 + \dots = \sum_{i=0}^{n-1} 2^i = \frac{2^{n-1+1} - 1}{2 - 1} = O(2^n).$$

MCD: MASSIMO COMUN DIVISORE

$$20, 30 \rightarrow 30-20, 20 \rightarrow 20-10, 10$$

$$10, 20$$

$$10, 10$$

$$\text{stop}$$

```
int MCD(int x, int y) {
```

$$D(x, y) = \Theta(1)$$

$$C(x, y) = \Theta(1)$$

$$a = 1$$

riduzione k variabile

CASO PEGGIORE:

$$MCD = 1$$

$$\begin{array}{l} 1, 8 \\ 3, 1 \\ 8, 1 \end{array}$$

```
// COND di TERM
if (x == y) return x;
```

// DECREASE + RECURSION + COMBINE

```
if (x > y) return MCD (x-y, y);
else return MCD (y-x, x);
```

$y-1 = 1$

$$T(1, y) = T(1, y-1) + 1 = 1 + 1 + 1 \dots = \sum_{i=0}^{y-1} 1 = y-1 = O(y).$$

DETERMINANTE MATRICE

$$M = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} \quad \text{sempre quadrata!}$$

$$\det(M) = (-1)^{1+1} \cdot (0) \cdot \det \begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix} + (-1)^{1+2} \cdot (1) \cdot \det \begin{bmatrix} 3 & 5 \\ 6 & 8 \end{bmatrix} + (-1)^{1+3} \cdot (2) \cdot \det \begin{bmatrix} 3 & 4 \\ 6 & 7 \end{bmatrix}$$

CASO ELEMENTARE: Mat 2×2 $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$

$$\det = 0 \cdot 3 - 1 \cdot 2$$

```
int det (int **mat, int N) {
```

```
    int det=0, **Sotto-mat;
```

// COND di TERMINAZIONE

```
if (N==2)
    return
    mat[0][0] * mat[1][1]
    - mat[0][1] * mat[1][0];
```

// RICORSIONE

```
for (int i=0; i < N; i++) {
    Sotto-mat = SottoMAT(mat, N, i);
    det += pow(-1, i) * mat[0][i] * det (Sotto-mat, N-1);
}
return det;
```

// FUNZ di SERVIZIO

```
int ** SottoMAT (int **mat, int N, int pos) {
```

```
    int ** sotto-mat = malloc ((N-1) * sizeof(int *));
    for (int i=0; i < N-1; i++) {
        sotto-mat[i] = malloc ((N-1) * sizeof(int));
```

```
        for (int i=1; i < N; i++) {
            for (int j=0; j < N; j++) {
                if (j < pos)
                    sotto-mat[i-1][j] = mat[i][j];
                else if (j > pos)
                    sotto-mat[i-1][j-1] = mat[i][j];
            }
        }
        return sotto-mat;
```

Prodotto di 2 int + KARATSUBA



4 moltiplicazioni ricorsive

$O(n^2)$ "QUADRATICA"

KARATSUBA

3 moltiplicazioni ricorsive

$O(n^{\log_2 3})$ "SUBQUADRATICA"



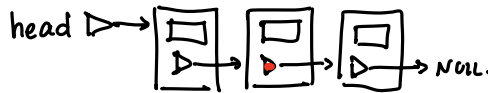
② LISTE e RICORSIONE

```
int LISTcount (link t) {  
    if (t == NULL) return 0;  
    return (1 + LISTcount (t->next));  
}
```

```
void LISTtraverse (link t) {  
    if (t == NULL) return;  
    ITEMdisplay (t->item);  
    LISTtraverse (t->next);  
}
```

invento ⇒ stampa al contrario

```
void LISTfree (link t) {  
    if (t == NULL) return;  
    LISTfree (t->next);  
    ITEMfree (t->item);  
    free (t);  
}
```



③ ALBERI BINARI e RICORSIONE

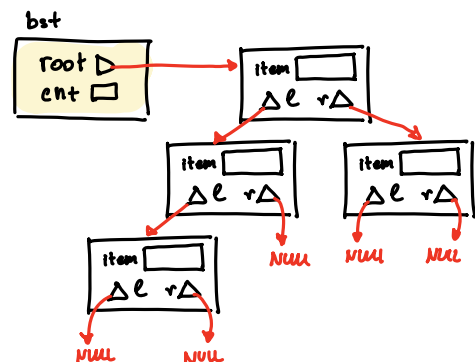
```
typedef struct node * link;
```

```
struct node {
```

```
    item item;  
    link r;  
    link l;  
}
```

```
struct bst {
```

```
    link root;  
    int cnt;  
}
```



```
int BSTcount (link t) {  
    if (t == NULL) return 0;  
    return (BSTcount (t->l) + BSTcount (t->r) + 1);  
}
```

```
int BSTheight (link t) {
```

```

if (t == NULL) return 0;

int h-left = BST height (t->l);
int h-right = BST height (t->r);

return MAX (h-left, h-right) + 1;
3

```

```

void BST preorder (bin t) {

```

```

    if (t == NULL) return;

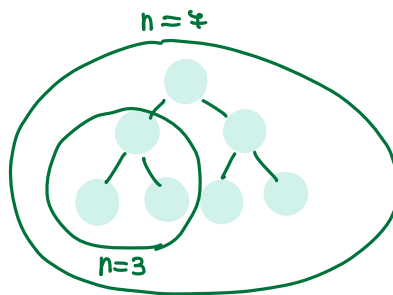
    ITEMdisplay (t->item);
    BST preorder (t->l);
    BST preorder (t->r);
3

```

ALBERO COMPLETO

EQ. RICORRENTE

$$\begin{aligned}
 D(n) &= \Theta(1) \\
 C(n) &= \Theta(1) \\
 a &= 2, \quad b = 2
 \end{aligned}$$

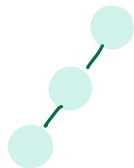


$$\begin{aligned}
 \Rightarrow T(n) &= 2T\left(\frac{n}{2}\right) + 1 \\
 T(1) &= 1
 \end{aligned}$$

$$\begin{aligned}
 \rightarrow T(n) &= 2[2[2T\left(\frac{n}{4}\right) + 1] + 1] + 1 = 1 + 2 + 4 + \dots = \\
 &= \sum_{i=0}^{\log_2 n} 2^i = \frac{2^{\log_2 n + 1} - 1}{2 - 1} = \Theta(n).
 \end{aligned}$$

ALBERO TOTALMENTE SBIANCATO $\equiv$ LISTA

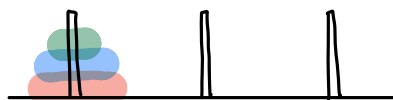
$$\begin{aligned}
 D(n) &= \Theta(1) \\
 C(n) &= \Theta(1) \\
 a &= 1, \quad b = 1
 \end{aligned}$$



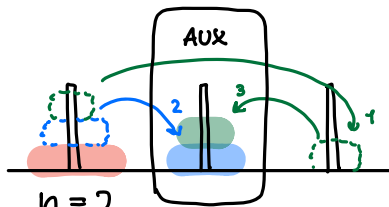
$$\begin{aligned}
 \Rightarrow T(n) &= T(n-1) + 1 \\
 T(1) &= 1
 \end{aligned}$$

$$\rightarrow T(n) = 1 + 1 + 1 + \dots = \sum_{i=0}^{n-1} 1 = \Theta(n).$$

TORRi di HANOI



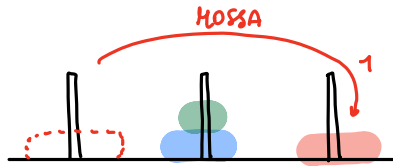
$n = 3$
 $src = 0$
 $dest = 2$



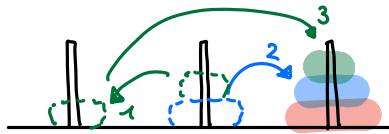
$n = 2$
 $src = 0$
 $dest = 1$

// CASI BANALI

$n = 1$	$n = 1$	$n = 1$
$src = 0$	$src = 0$	$src = 2$
$dest = 2$	$dest = 1$	$dest = 1$



$n = 1$
 $src = 0$
 $dest = 2$



$n = 2$
 $src = 1$
 $dest = 2$

$n = 1$	$n = 1$	$n = 1$
$src = 1$	$src = 1$	$src = 0$
$dest = 0$	$dest = 2$	$dest = 2$

```
void Hanoi (int n, int src, int dest) {
```

```
    int aux = 3 - (src + dest);
```

```
    // CASO BANALE
```

```
    if (n == 1) {
```

```
        printf("sposta disco da piolo %d a %d", src, dest);
```

```
        return;
```

```
    }
```

```
    // RICORSIONE
```

```
    Hanoi (n-1, src, aux);
```

```
    printf("sposta disco da piolo %d a %d", src, dest);
```

```
    Hanoi (n-1, aux, dest);
```

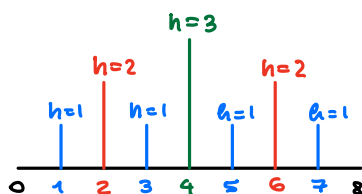
```
}
```

$$\left. \begin{array}{l} D(n) = \Theta(1) \\ C(n) = \Theta(1) \\ a = 2, \quad r_1 = 1 \end{array} \right\} \begin{array}{l} T(n) = 2T(n-1) + 1 \\ T(1) = 1 \end{array}$$

$$\begin{aligned} \rightarrow T(n) &= 2[2[2T(n-3) + 1] + 1] + 1 = 1 + 2 + 4 + \dots = \\ &= \sum_{i=0}^{n-1} 2^i = \frac{2^{n-1+1} - 1}{2 - 1} = O(2^n). \end{aligned}$$

IL RIGHELLO

$(0, 8) \rightarrow$



TRACCIARE UNA
 RIGA ALTA h
 CON LA FUNZIONE
 $mark(pos, h)$

```
void ruler (int l, int r) {
```

```
    int h = log2(r - l + 1);
```

```
int m = (l+r)/2;
//TERMINAZIONE
if (l==o) return;
```

```
// PASSI RECORSIONE
ruler (l, m);
mark(m, l);
ruler (m, r);
}
```

```
void mark(int pos, int l){
    printf("%d\t", pos);
    for(int i=0; i<l; i++)
        printf("-");
    printf("\n");
}
```

```
1  —
2  — —
3  —
4  — — —
5  —
6  — —
7  —
```

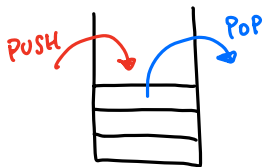
$D(n) = \Theta(1)$
 $C(n) = \Theta(1)$
 $a=2, b=2$

$\Rightarrow O(n)$

(E) STACK FRAME

Meccanismo per gestire chiamate annidate e ritorni

⇒ STACK



- PUSH

- POP

- gestione LIFO "Last In First Out"

STACK FRAME (o record di attivazione)

Struttura dati in RAM che contiene tutti i dati necessari alla gestione di una funzione:

1. PARAMETRI FORMALI
2. VARIABILI LOCALI
3. @ di RITORNO
4. PUNTATORE AL CODICE

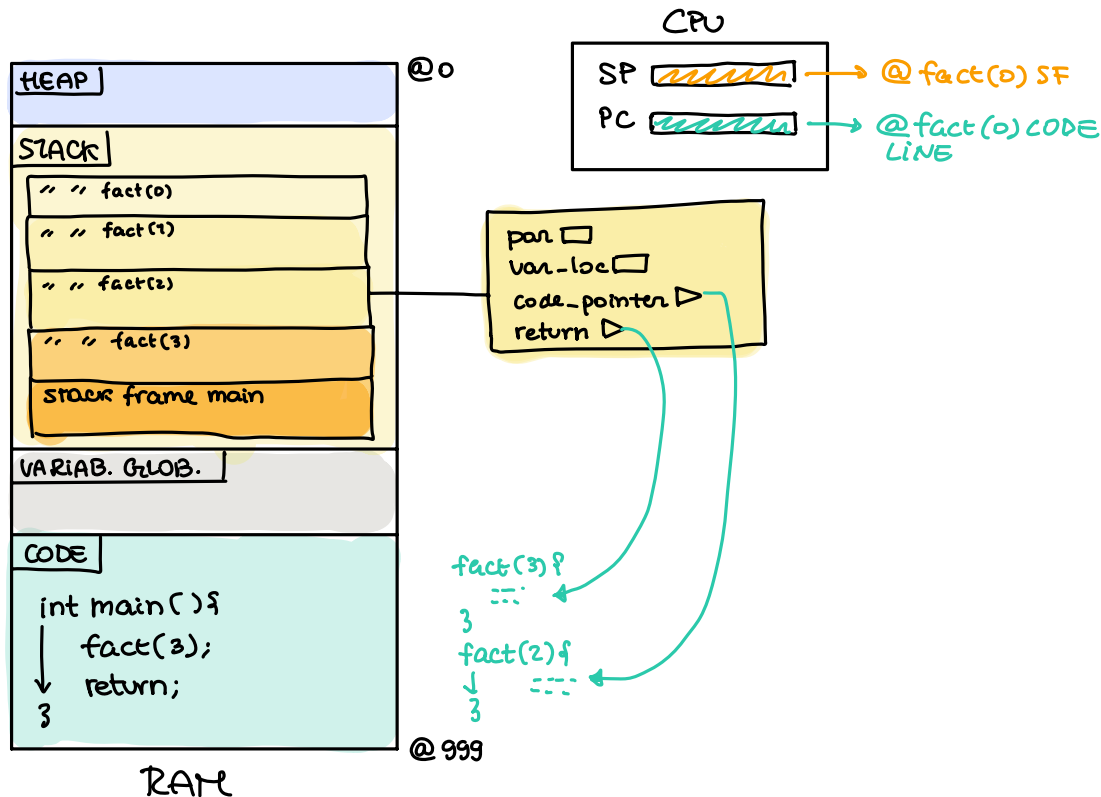
Viene creato alla chiamata di una funzione e distrutto al suo termine.

STACK di SISTEMA

Struttura dati in RAM che contiene tutti gli STACK FRAME.
 Si genera a partire da @ maggiori verso @ minori ↑.

⇒ Se oltrepassa lo spazio dedicato: **STACK OVERFLOW**

L'@ del primo stack frame disponibile e' nel registro CPU chiamato **STACK POINTER**



⑥ TAIL vs NON TAIL-RECURSIVE

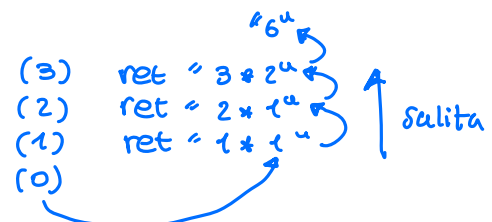
- TAIL** = chiamata ricorsiva ultima da eseguire prima del `return`:

`return FUNZ-R(...);` ⇒ calcoli in discesa

- NON TAIL** = non ultima:

`return n * FUNZ-R(...);` ⇒ calcoli in risalita

```
int fact_nontailR(int n) {
    if (n == 0) return 1;
    return n * fact_nontailR(n);
}
```



main: n, 1

```
int fact_tailR(int n, int fact){
    if(n==0) return fact;
    return fact_tailR(n-1, n*fact);
}
```

(3, 1)
(2, 3)
(1, 6)
(0, 6)

discesa

ret "6"

⊕ PROS TAIL :

- + efficienti
- no multi stack:
rimpiazza lo stack della chiamata prec
- ⇒ no stack overflow
- trasform diretta in iterative

⊖ CONS TAIL :

- difficili da scrivere

⑥ DUAITA' RICORSIONE - ITERATIONE

Le native iterative sono più efficienti

// NATIVA

```
int fib_IT(int n){
    if(n==0 || n==1)
        return n;

    int f2=0;
    int f1=1;
    int f=f1+f2;
    for(int i=3; i<=n; i++){
        f2=f1;
        f1=f;
        f=f1+f2;
    }
    return f;
}
```

// NATIVA

```
int fact_IT(int n){
    if(n==0) return 1;

    int fact=1;
    for(int i=1; i<=n; i++)
        fact *= i;
    return fact;
}
```

vs

// NON TAIL-RECURSIVE

```
int fact_IT_STACK(int n){
    int fact=1;
    STACK stack = STACKinit(n);

    while(n>0){
        STACKpush(stack, n);
        n--;
    }

    while(STACKsize(stack)>0){
        n = STACKpop(stack);
        fact *= n;
    }
}
```

```
    return fact;
```

```
}
```

- **RICERCA**:
 - S = spazio delle soluzioni possibili
 - V = spazio delle soluzioni valide
 - $V \subseteq S$
 - OB {
 1. Appurare se $V = \emptyset$
 2. Elenicare almeno 1 el di V
 - ES:
 - Insieme delle parti che soddisf una condizione
 - 8 regine
 - Sudoku
 - Cammini semplici da un vertice in G
- **OTTIMIZZAZIONE**: $S \equiv V$: tutte le sol sono valide
 - OB { f : funzione obiettivo da minimizzare o massimizzare
 - ES:
 - Max oggi compatibili con contenitore
 - Cammini semplici a lung max in G

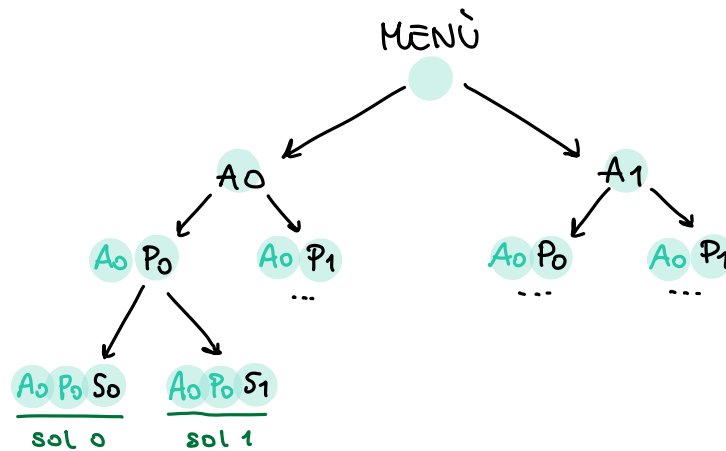
(A) TIPI DI RICERCA

Struttura dati per contenere una possibile soluzione:

- **RIC. in AMPIEZZA** $\leftrightarrow$ coda, politica FIFO
- **RIC. in PROFONDITÀ** $\leftrightarrow$ pila, politica LIFO
- **RIC. BEST-FIRST** $\leftrightarrow$ coda a priorità

Albero di ricerca :

- RADICE : sol nota iniz
- ALTEZZA h : dim della sol
- GRADO κ : n° max di scelte poss
- NODI INTERMEDI : sol parziali
- FOGLIE : sol $\rightarrow$ si determina se sono valide o meno



⑧ CALCOLO COMBINATORIO per S

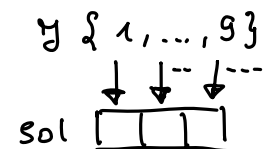
S : raggruppamenti di elementi di un gruppo secondo regole di associazione

- elementi **DISTINTI** o no
- ——— **ORDINATI** o no
- ——— **RIPETUTI** o no

• PRINCIPIO di ADDIZIONE

Insiemi disgiunti : 4 corsi di inform e 5 di mate,
he devo scegliere 3

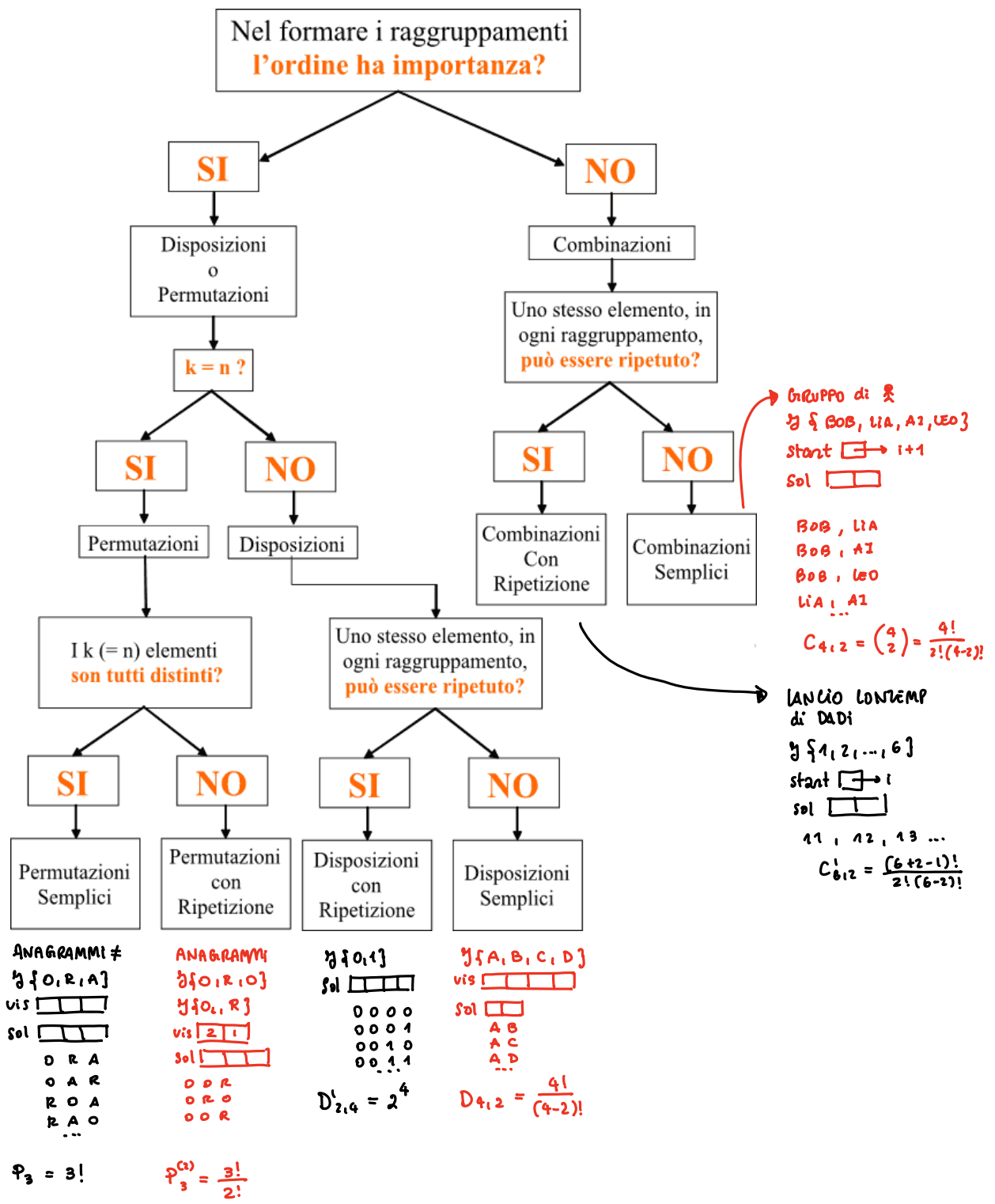
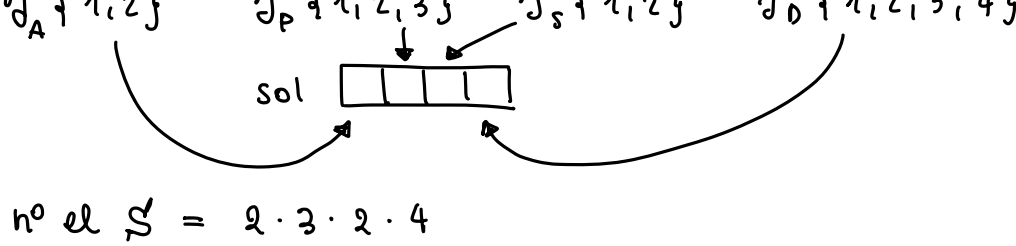
$$n^{\circ} \text{ di } S = 9 \cdot (9-1) \cdot (9-2)$$



• PRINCIPIO di MOLTIPLICAZIONE

Menu : 2 antipasti , 3 primi , 2 secondi e 4 dolci ,
modi di comporre il menu

$$M = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100\}$$



INSIEMI MASSIMALI e MINIMALI

MAX → sol → dim

--

 1 → 2 → 3 → 4 FINCHÉ NON POSSO PIÙ

MIN → sol → dim

--

 n → n-1 → n-2 FINCHÉ NON POSSO PIÙ

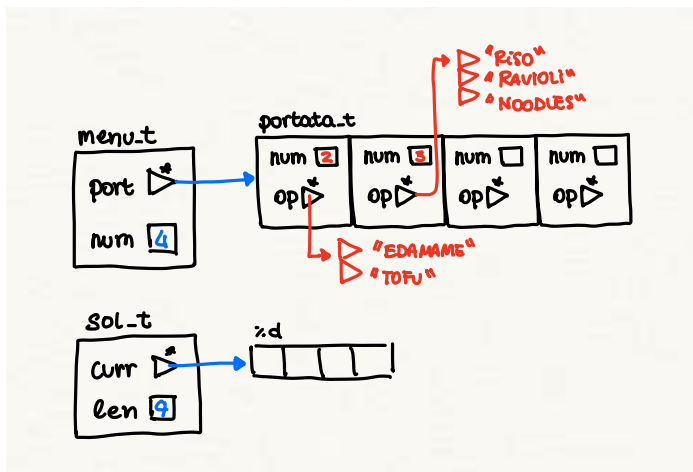
$$n^o \text{ el } S : \sum_{i=0}^x (\text{modello_combinatorio}(i))$$

$$\sum_{i=x}^y (\text{modello_combinatorio}(i))$$

② CALCOLO COMBINATORIO : CODICE

• PRINCIPIO di MOLTIPLICAZIONE

Menu: 2 ant, 3 pr, 2 sec, 4 dol



```
typedef struct portata {
    char **op;
    int num;
} portata_t;
```

```
typedef struct menu {
    portata_t *port;
    int num;
} menu_t;
```

```
void princ_molt ( menu_t menu, sol_t *sol, int pos ) {
```

```
    // COND TERM
```

```
    if ( pos >= sol->len ) {
```

```
        // ...
        return;
    }
```

```
    // PASSO RICORSIONE
```

```
    int num = menu.port[pos].num;
```

```
    for ( int i = 0; i < num; i++ ) { // indici di op di port pos-esima
```

```
        sol->cum[pos] = ...
```

```

    ↓
    3
    ↓
    princ_molt (menu, sol, pos+1);
    }

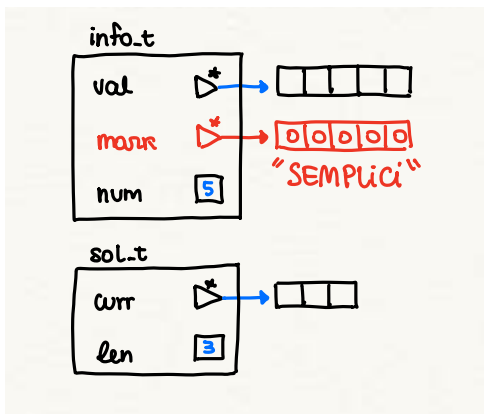
```

BACKTRACKING

Annulla l'ultima scelta effettuata

↳ dopo la chiamata
della funz_R

• DISPOSIZIONI o PERMUTAZIONI



```
void disp (info_t info, sol_t * sol, int pos) {
```

// COND TERM

```
if (pos >= sol->len) {
```

...

```
return;
```

```
}
```

// PASSO RICORSIONE

```
for (int i=0; i < info.num; i++) {
```

```
    if (info.mark[i] == 0) {
```

```
        info.mark[i] = 1;
```

```
        sol->curr[pos] = i;
```

```
        disp (info, sol, pos+1);
```

```
        info.mark[i] = 0;
```

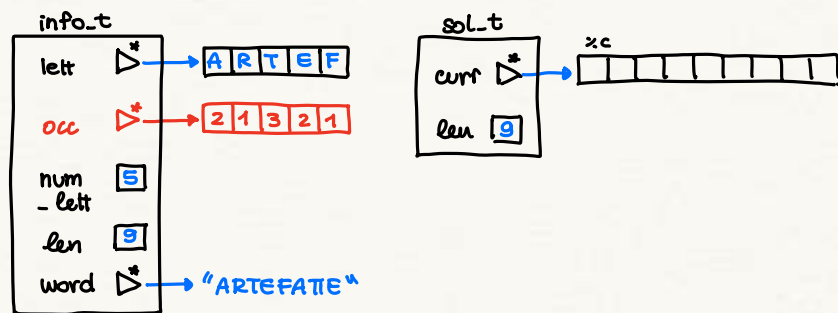
```
    }
```

```
}
```

```
}
```

ANAGRAMMA con LETTERE RIPLEUTE - ottimizzazione -

"ARTEFATTE"



```
void perm_rep (info_t info, sol_t * sol, int pos) {
```

// COND di TERM

```
if (pos >= sol->len) {
```

```
    ...
    return;
}
```

// PASSO RICORSIONE

```
for (int i=0; i < info.num_lett; i++) {
```

```
    if (info.occ[i] > 0) {
```

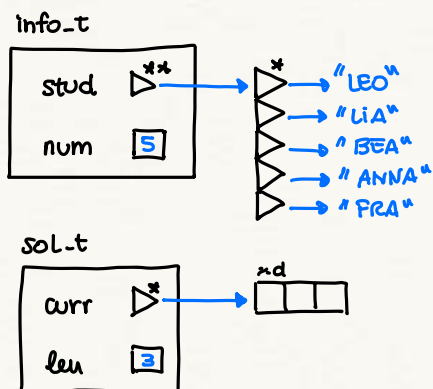
```
        info.occ[i]--;
        sol->curr[pos] = lett[i];
        perm_rep (info, sol, pos+1);
        info.occ[i]++;
    }
```

```
}
```

```
}
```

• COMBINAZIONI SEMPLICI

GRUPPO di PERSONE



```
void comb_semp (Info_t info, sol_t * sol, int start, int pos) {
```

// COND TERM

```
if (pos >= sol->len) {
```

```
    ...
    return;
}
```

// PASSO RICORSIONE

```

// PASSO RICORSIONE
for (int i = start; i < info.num; i++) {
    sol[pos] = i;
    comb_rep(info, sol, i+1, pos+1); // "SEMPLICI"
}
}

```

• COMBINAZIONI RIPEUTE

LANCIO CONTEMP DI 3 DADI 

$y \in \{1, 2, 3, 4, 5, 6\}$

sol 

```

void comb_rep(sol_t *sol, int start, int pos) {

```

```

    // COND TERM
    if (pos >= sol->len) {

```

```

        return;
    }

```

```

    // PASSO RICORSIONE

```

```

    for (int i = start; i <= 6; i++) {

```

```

        sol->arr[pos] = i;

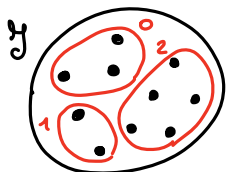
```

```

        comb_rep(sol, i, pos+1); // "RIPEUTE"
    }
}

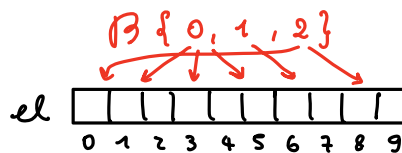
```

④ PARTIZIONAMENTO - CODICE



$n=10$ elementi
partitionamento
in $k=3$ blocchi

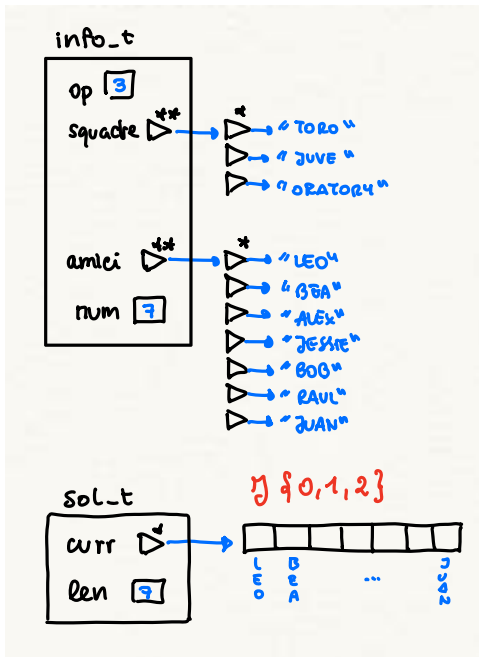
rappresentazione →



- OB {
1. Determinare una qualsiasi partizione che rispetti una condizione
→ DISPOSIZIONI CON RIPETIZIONE
 2. Determinare una partizione su k blocchi esatti
→ ALGORITMO di ER
 3. Insieme maximale
→ COMBINAZIONI SEMPLICI con $sol \rightarrow len \wedge$
→ DISP RIP con $\{0,1\}$ (memo efficiente)

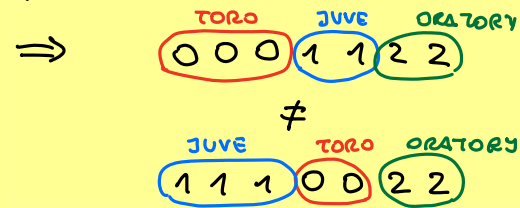
• QUALSIASI PARTIZIONE: D'

"GRUPPI di PERSONE DA FAR ENTRARE IN UNA SPECIFICA SQUADRA"



E' importante a quale specifico gruppo appartengono le persone!

Non solo quali raggruppamenti posso formare!



void disp_rep (info_t info, sol_t * sol, int pos) {

//COND TEAM

if (n >= 0) {

for (int i = 0; i < info.op; i++) { // & squadra

printf("SQUADRA %s: ", info.squadre[i]);

for (int j = 0; j < sol->len; j++) { // & amico

if (sol->curr[j] == i) { // se e' alla squadra

printf("%s, ", info.amici[j]);

}

printf("\n");

}

printf("\n");

}

return;

}

//PASSO RICORSIONE

for (int i = 0; i < info.op; i++) {

sol->curr[pos] = i;

disp_rep (info, sol, pos + 1);

}

}

>> SQUADRA TORO: BEA, LEO...

>> SQUADRA JUVE: JUAN,...

>> SQUADRA ORATORY: ALEX,...

🕒 POWERSET : caso particolare dove $\mathcal{M} \{0,1\}$

"0" = elemento non incluso

"1" = elemento incluso

sol[pos] = 1;
//AGGIORNO
powerset (... , pos + 1);

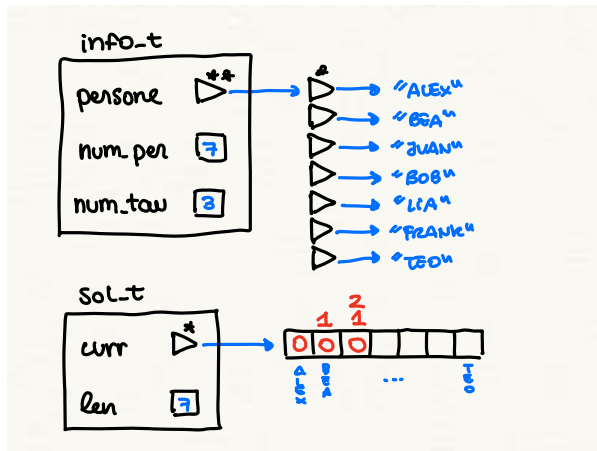
$p() = \text{INSIEME delle PARTI} = \text{POWERSET}$
di $\{0, 1, 2\}$

$\{\} \{0\} \{1\} \{2\} \{01\} \{02\} \{12\} \{012\}$

$\text{sol}[\text{pos}] = 0;$
// BACKTRACK
 $\text{powerset}(\dots, \text{pos}+1);$

• ALGORITMO di ER (1987)

"GRUPPI di PERSONE a TAVOLA"



Importa SOLO come si formano i gruppi!

Non conta a quale gruppo specifico appartengono le persone!

$\Rightarrow$ 0 0 0 1 1 2 2

coinciderebbe con

1 1 1 0 0 2 2

nel caso delle D'!

Inutile spreco di risorse

```
void Er_algorithm (info-t info, sol-t * sol, int div_, int pos){
```

//COND TERM

```
if (pos >= sol->len){
```

```
    if (div_ == info.num_tav){
```

```
        STAMPA gruppi (info, sol, div_);
```

```
    }
```

```
    return;
```

```
}
```

//PASSI RICORSIONE

```
for (int i=0; i < div_; i++){
```

```
    sol->curr[pos] = i;
```

```
    Er_algorithm (info, sol, div_, pos+1);
```

```
}
```

```
sol->curr[pos] = div_;
```

```
Er_algorithm (info, sol, div_+1, pos+1);
```

```
}
```

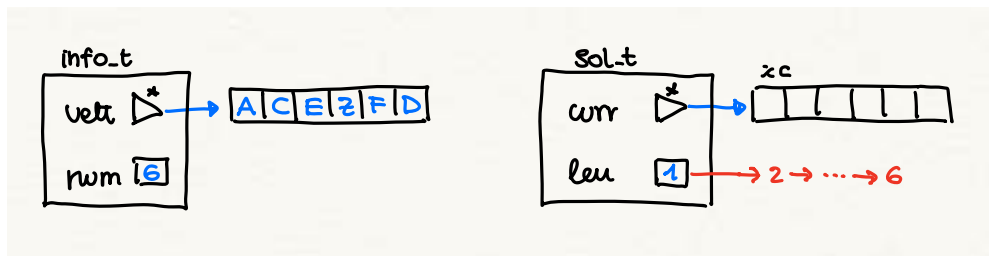
- FILTRO PER TROVARE SOLO PARTIZIONI DA K GRUPPI!

- NUMERO di GRUPPI PARABILI AL PASSO POS-ESIMO!



UNICO METODO CHE TROVA PARTIZIONI DI ESATTAMENTE (K) GRUPPI!

• INSIEME MASSIMALE



```
void ins_massimale (info_t info, sol_t * sol) {
```

```
    int trovato = 1;
    int pos = 0;
```

```
    // ACCRESCO LA DIM di SOL FINCHÉ TROVO SOL COMPATIBILI
    for (int i = 1; i <= info.num && trovato; i++) {
```

```
        trovato = 0;
        sol->len = i;
        funzione_R (info, sol, &trovato, pos);
```

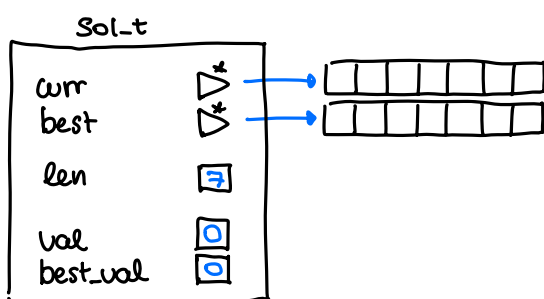
```
    }
```

Ⓜ POWERSET : ulteriore implementazione

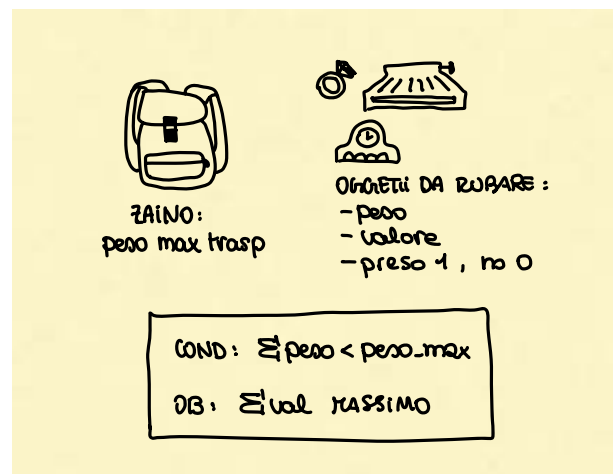
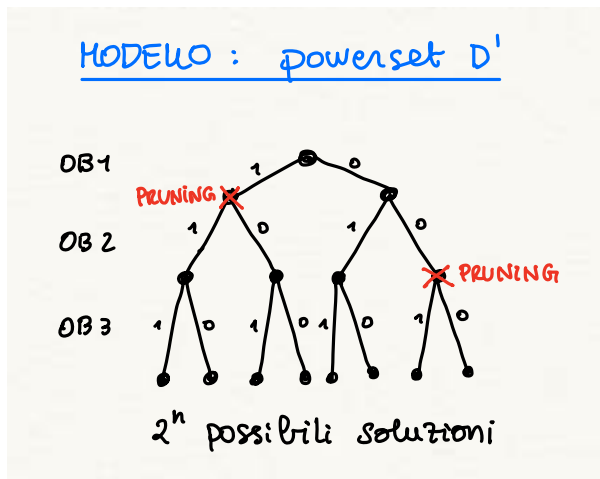
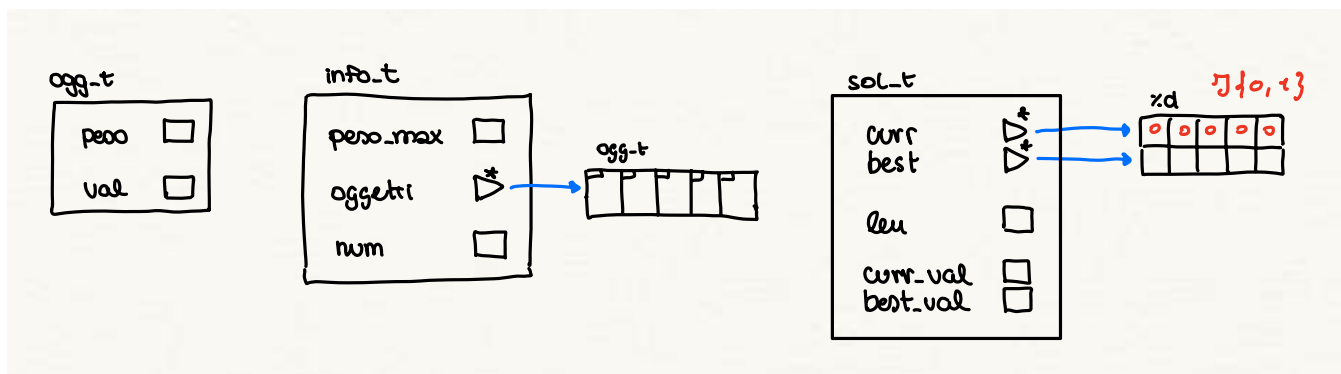
$sol \rightarrow len = 0 \rightarrow n$

ⓔ PROBLEMI di OTTIMIZZAZIONE

Si esplora tutto lo spazio delle sol S e si tiene traccia della sol ottima



• ZAINO DISCRETO "0/1 knapsack" $O(2^n)$



```
void powerset ( info_t info, sol_t * sol, int wt_curr, int pos ) {
```

//COND TERM

```
if ( pos >= sol->len ) {
```

```
    if ( sol->curr_val > sol->best_val ) {
```

```
        ↓ ...
```

```
        return;
```

```
    }
```

//PASSO RICORSIONE

```
ogg_t ogg = info.oggetti[pos]; *
```

```
for ( int i=1; i >= 0; i-- ) {
```

```
    if ( i==1 ) { // prendo
```

```
        //PRUNING
```

```
        if ( wt_curr + ogg.peso <= info.wt_max ) {
```

```
            sol->curr[pos] = i;
```

```
            sol->curr_val += ogg.val; //UPDATE
```

```
            powerset ( info, sol, wt_curr + ogg.peso, pos+1 );
```

```
            sol->curr_val -= ogg.val; //BACKTRACK
```

```
        }
```

```
    } else { //lascio
```

```
        sol->curr[pos] = i;
```

```
        powerset ( info, sol, wt_curr, pos+1 );
```

```
    }
```

```
}
```


// PRUNING

```
if ( wt_curr + ogg peso <= info.wt-max ) {
```

11 PRENDO

sol $\rightarrow$ `wrr[pos] = 1;`

Sol $\rightarrow$ `wrr_val += ogg.val; //UPDATE`

powerset (---, wt-curr + ogg. peso, ---)

///ascio

sol $\rightarrow$ curr [pos] = 0;

```
sol → wvr_val = ogg_val; // BACK
```

```

powerset [-1, pos + 1);

```

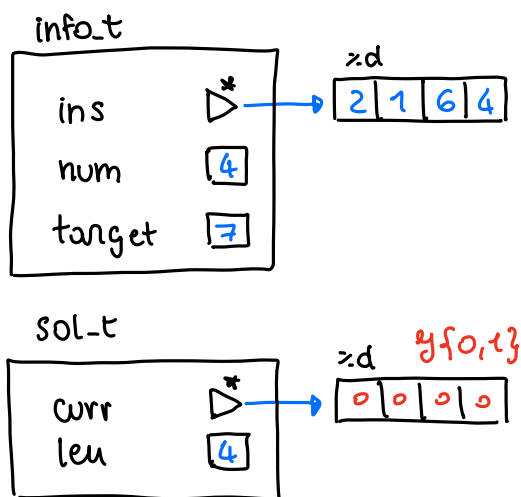
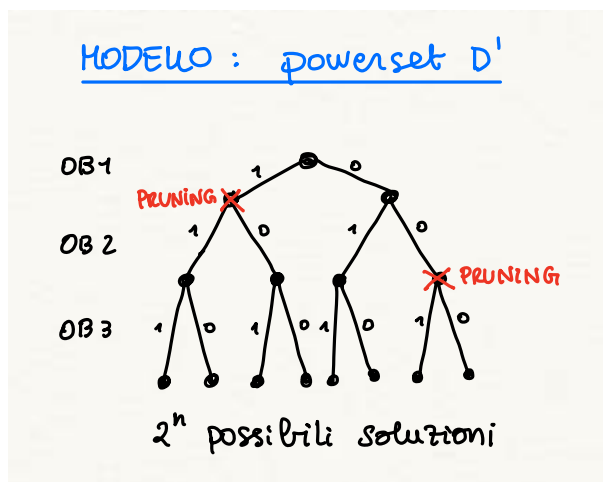
```
    } else {
```

↓ powerset(..., pos+1);

3

- SOMMA SOTTO-INSIEMI $\mathcal{O}(2^n)$

S of $\{2, 1, 6, 4\} \rightarrow$ solution t.c $\exists! \text{el} \in \text{set} = \neq$

$$\Rightarrow \{2, 1, 4\}, \{1, 6\}$$


```
void sumset (info-t info, sol-t *sol, int sum, int pos) {
```

// COND TERM

if (pos >= sol → len) {

```
if (sum == info.target)
```

```
STAMPA sol(sol);
```

```
↓ return;
```

3

// PASSO RICORSIONE

```
int num = info.ins[pos];
```

// PRUNING

```
if (sum + num <= info.target) {
```

```
sol[pos] = 1; // prendo
```

```
sumset (info, sol, sum + num, pos + 1);
```

Sol [pos] = 0; // la xio

```
sumSet (info, sol, sum, pos+1);
```

```

} else {

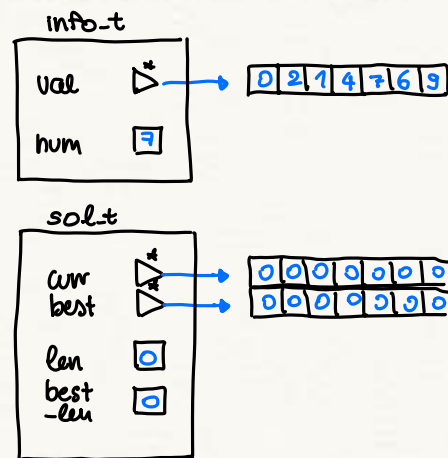
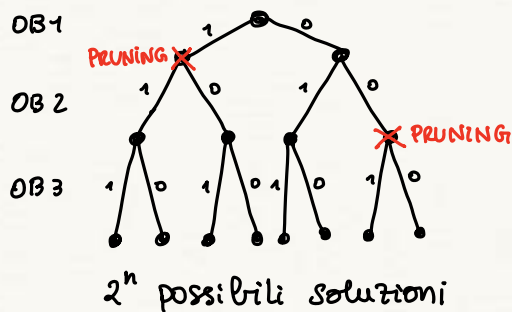
```

↓ sum-set (info, sol, sum, pos+1);

• LIS RICORSIVA "Longest Increasing Sequence"

$S = \{0, 2, 1, 4, 7, 6, 9\} \rightarrow \{0, 2, 4, 7, 9\} \quad n=5$

MODELLO: powerset D'



```
void Lis-R (info-t info, sol-t * sol, int last, int pos) {
```

//COND. TERM

```
if (pos >= info.num) {
    if (sol->len > sol->best-len) {
        // 😊
    }
    return;
}
```

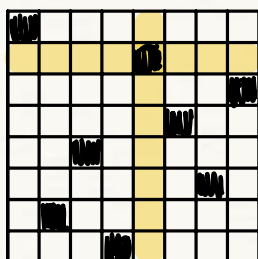
//PASSO RICORSIONE

```
if (pos == 0 || info.val[pos] > last) {
    sol->curr[pos] = 1;
    sol->len++;
    Lis-R (info, sol, info.val[pos], pos+1);

    sol->curr[pos] = 0;
    sol->len--;
    Lis-R (info, sol, last, pos+1);
} else {
    Lis-R (info, sol, last, pos+1);
}
```

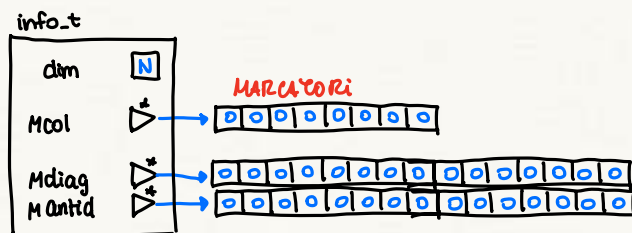
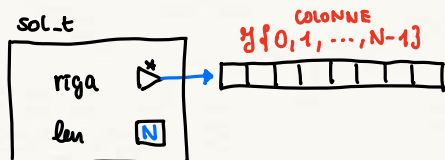
• 8 REGINE

Scacchiera $N \times N$: disporre N regine t.c. no scacco reciproco

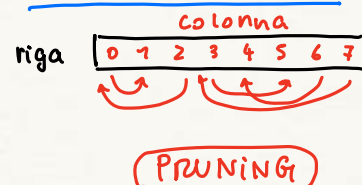


Ogni RIGA, N
 COLONNA, N
 DIAG, $2 \times N - 1$
 ANTI DIAG, $2 \times N - 1$

contiene 1 e 1 sola REGINA!



MODELLO : $P = n!$



```
void perm_semp(info_t info, sol_t *sol, int r) {
```

// COND TERM

```
if (r >= sol->len) {
    STAMPA sol(info, sol);
    return;
}
```

// PASSO RIDUZIONE

```
for (int c = 0; c < N; c++) { // colonna
```

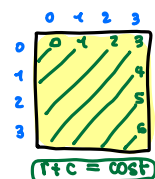
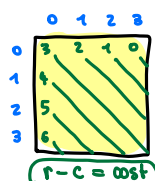
// PRUNING

```
if ( info.Mcol[c] == 0 &&
    info.Mdiag[r-c+(N-1)] == 0 &&
    info.Mantid[r+c] == 0 ) {
```

```
    sol->curr[r] = c;
    info.Mcol[c] = 1;
    info.Mdiag[r-c+(N-1)] = 1;
    info.Mantid[r+c] = 1;
```

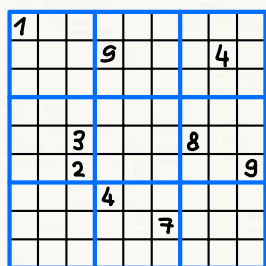
```
    perm_semp(..., r+1);
```

```
    info.Mcol[c] = 0;
    info.Mdiag[r-c+(N-1)] = 0;
    info.Mantid[r+c] = 0;
}
```



• SUDOKU

Griglia $9 \times 9 \rightarrow \{1, 2, \dots, 9\}$ su celle vuote



MODELLO : D'

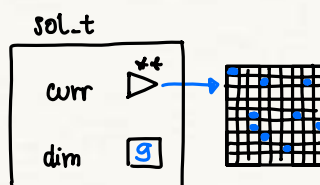
κ celle libere

$\{1, 2, \dots, 9\}$



PRUNING

$\Rightarrow D'_{9,\kappa} = 9^{\kappa}$



```
void disp_rep (info_t info, sol_t * sol, int pos) {
```

```
    // COND TERM
```

```
    if (pos >= sol->dim * sol->dim) {
        ↓
        STAMPA(sol);
        return;
    }
    ↓
```

```
    // PASSO RICORSIONE
```

```
    // COORDINATE in MAT
```

```
    int i = pos / sol->dim;
```

```
    int j = pos % sol->dim;
```

```
    if (sol->curr[i][j] != 0) { // cella occup
```

```
        ↓
        disp_rep(..., pos+1);
    }
    ↓
```

```
    else { // cella libera
```

```
        for (int k = 1; k <= sol->dim; k++) { // g {1, ..., 9}
```

```
            if (CHECKINS(sol, i, j, k)) { // RUNNING
```

```
                ↓
                sol->curr[i][j] = k;
```

```
                disp_rep(..., pos+1);
            }
        }
    }
}
}
}

```

```
int CHECKINS(sol_t * sol, int i, int j, int val) {
```

```
    // CHECK righe & colonne
```

```
    for (int k = 0; k < sol->dim; k++)
```

```
        if (sol->curr[i][k] == val ||
```

```
            sol->curr[k][j] == val)
```

```
            return 0;
```

```
    // CHECK regione
```

```
    int n = sqrt(sol->dim);
```

```
    int i_start = (i/n) * n;
```

```
    int j_start = (j/n) * n;
```

```
    for (int k = 0; k < n; k++)
```

```
        for (int l = 0; l < n; l++)
```

```
            if (sol->curr[i_start+k][j_start+l] == val)
```

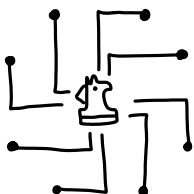
```
                return 0;
```

```
    return 1;
```

```
}
```

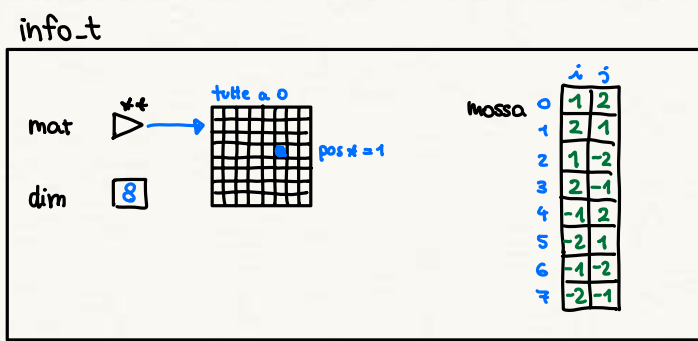
• TOUR DEL CAVALLO

Sequenza di mosse t.c. ogni cella visitata 1 e 1 sola volta



⇒ a ogni passo posso provare 8 MOSSE

⇒ termino quando ho coperto tutte le celle



```

void tour_cavallo (info-t info, int src-i, int src-j, int *ok-p, int cnt) {
    info.mat[i][j] = cnt; // visit & mark

    // COND TERM
    if (cnt == info.dim * info.dim + 1) {
        (*ok-p) = 1;
        STAMPA SOL (info);
        return;
    }

    // PASSO RICORSIONE
    for (int k = 0; k < 8; k++) {
        int new-i = i-src + mossa[k][0];
        int new-j = j-src + mossa[k][1];

        if ((new-i < info.dim) && (new-i >= 0) &&
            (new-j < info.dim) && (new-j >= 0)) { // limiti
            if (info.mat[new-i][new-j] == 0) { // !visited yet
                tour_cavallo (info, new-i, new-j, ok-p, cnt+1);
                if ((*ok-p) == 1)
                    return;
            }
        }
    }

    info.mat[src-i][src-j] = 0; // backtrack
    return;
}
  
```

TOUR CAVALLO $\equiv$ CAMMINO DI HAMILTON

- vertici = caselle
- archi = tra coppie di vertici mutualmente raggiungibili

G non orientato

$\mathcal{O}(2^n)$ - ESPONENZIALE

